

EECS 482

Introduction to operating systems

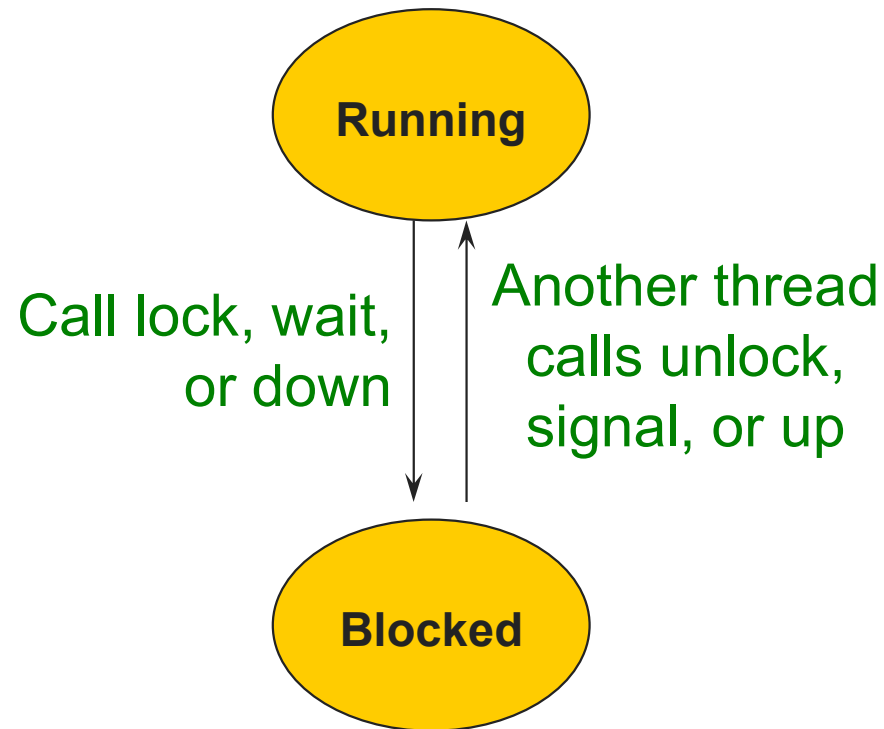
Switching threads; creating threads

Peter Chen
University of Michigan

Two perspectives when sharing a processor

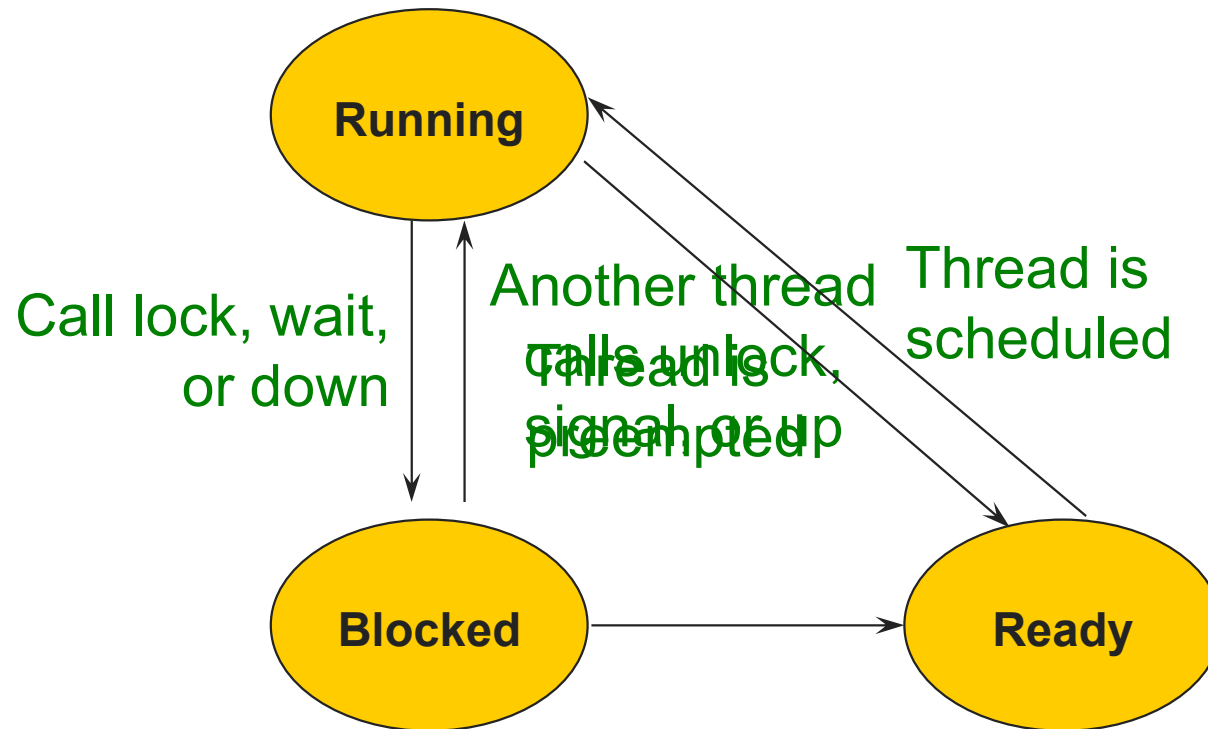
- Thread view:
 - Running → Paused → Running
- CPU view:
 - Thread 1 → Thread 2 → Thread 1
- First, let's take the thread view

States of a thread



What if there are more threads than CPUs?

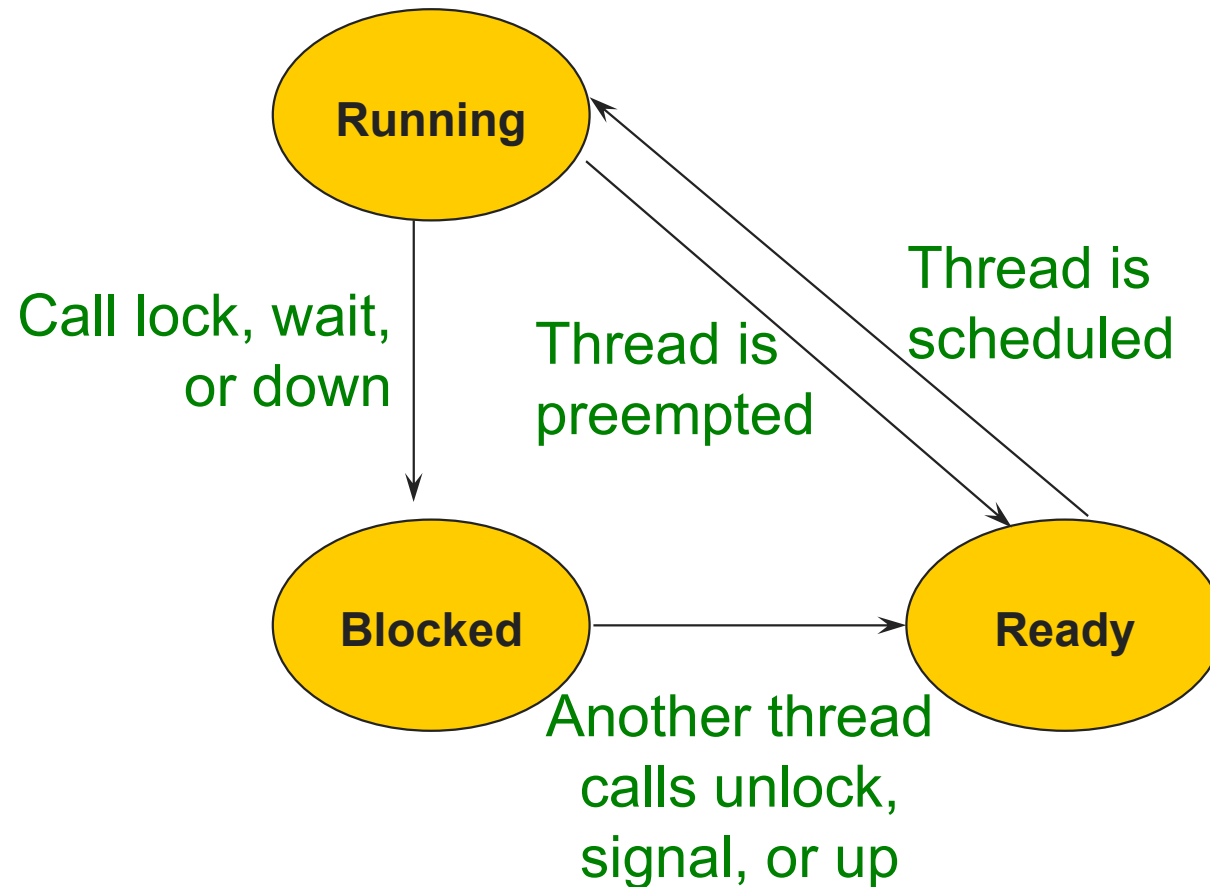
States of a thread



Why no transition from Blocked to Running?

Why no transition from Ready to Blocked?

States of a thread



Thread control block (TCB)

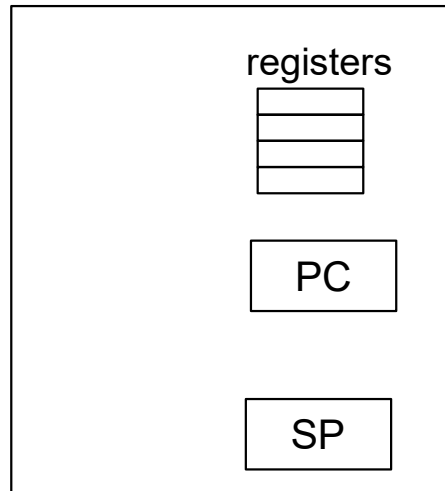
- Per-thread OS data structure for thread info
 - Stores thread context when thread is not running
- For each state of a thread, where is the thread context?
 - Running:
 - Ready:
 - Blocked:

Two perspectives when sharing a processor

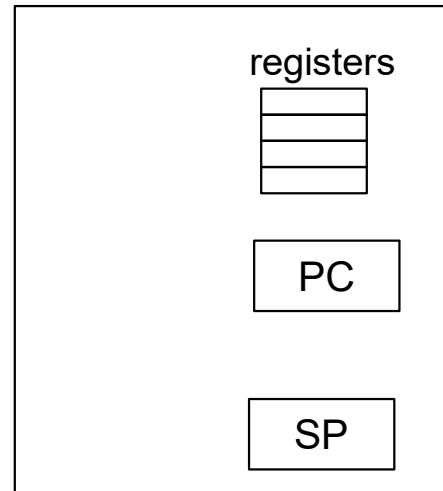
- Thread view:
 - Running → Paused → Running
- CPU view:
 - Thread 1 → Thread 2 → Thread 1
- Next, let's take the CPU view

Sharing the CPU

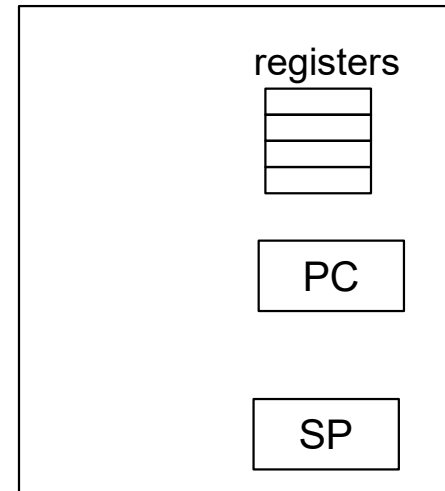
Thread context



Thread context



Thread context



CPU

Switching threads

- 
1. Current thread returns control to OS
 2. OS chooses new thread to run
 3. OS saves current thread state: CPU to TCB
 4. OS loads context of next thread: TCB to CPU
 5. OS runs next thread

How does thread return control to OS?

- **Internal events**
 - Thread calls `lock()`, `wait()`, `down()`, etc.
 - Thread requests OS to do some work (e.g., I/O)
 - Thread voluntarily gives up CPU with `yield()`
- **Are internal events sufficient?**
- **External events** (initiated by something outside the thread)
 - Interrupts: a hardware event that transfers control from thread to OS interrupt handler

Interrupts

- Hardware events (implemented by CPU)
 1. Some hardware component (e.g., I/O device) raises an interrupt
 2. Processor checks for pending interrupt after each instruction
 3. When pending interrupt is detected, CPU stops executing current code and calls OS **interrupt handler** (address of interrupt handler is stored in **interrupt vector table**)
- Example: timer interrupt
 - OS could set timer to go off every 10 ms
 - Guarantees that OS will get control back in ≤ 10 ms

Switching threads

- ➔ 1. Current thread returns control to OS
- 2. OS chooses new thread to run
- 3. OS saves current thread state: CPU to TCB
- 4. OS loads context of next thread: TCB to CPU
- 5. OS runs next thread

Choosing next thread to run

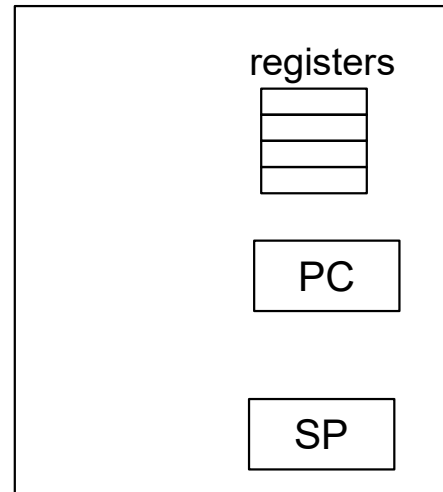
- 1 ready thread: run the ready thread
 - What if there's only one thread, and that thread calls yield?
- >1 ready thread: need to make a decision
 - OS scheduling policy
 - Lots of options: FIFO, priority, round robin, etc.
- 0 ready threads: what should CPU do?

Switching threads

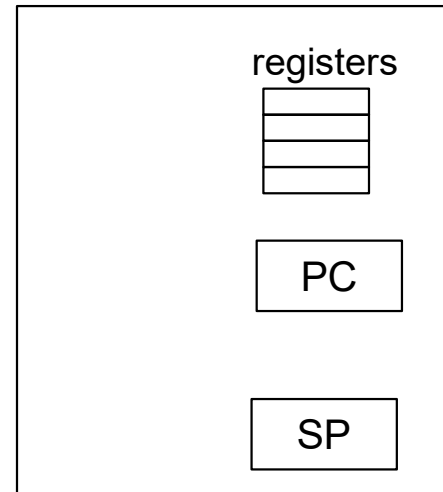
1. Current thread returns control to OS
- ➔ 2. OS chooses new thread to run
3. OS saves current thread state: CPU to TCB
4. OS loads context of next thread: TCB to CPU
5. OS runs next thread

Save state of the current thread (copy from CPU to memory)

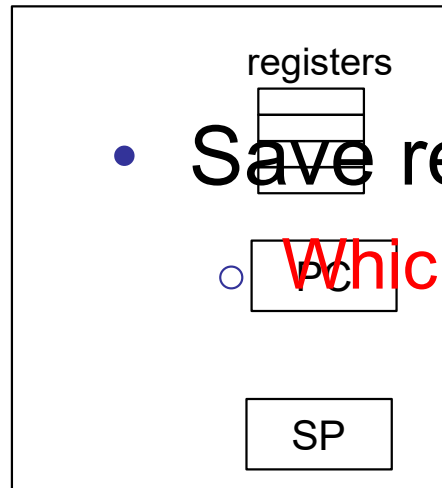
Thread context



Thread context



Thread context



CPU

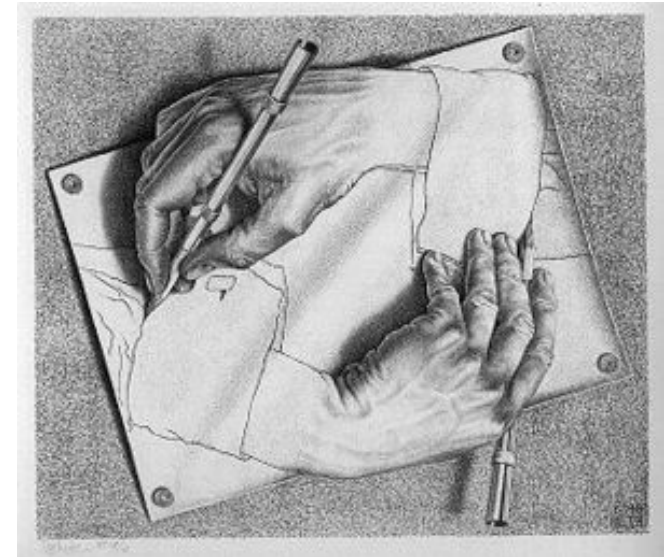
- Save registers, stack pointer, PC
 - Which instructions (from EECS 370)?

Save state of the current thread (copy from CPU to memory)

- Tricky to get right!

- Why won't the following code work?

```
100    save PC
101    switch to next thread
```



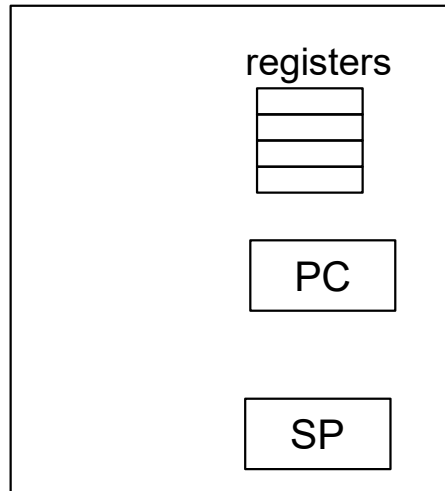
- Involves careful assembly-language code
- Linux: `getcontext()` copies context from CPU to memory

Switching threads

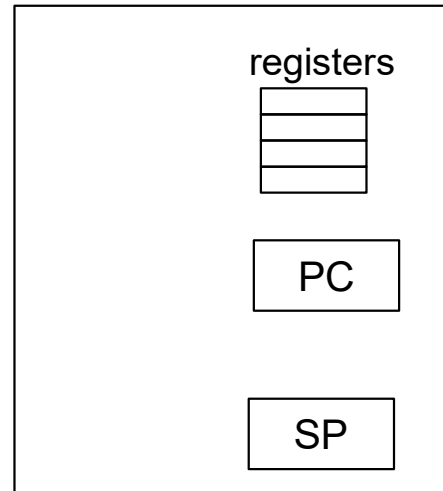
1. Current thread returns control to OS
2. OS chooses new thread to run
- ➔ 3. OS saves current thread state: CPU to TCB
4. OS loads context of next thread: TCB to CPU
- ➔ 5. OS runs next thread

Load state of the next thread (copy from memory to CPU)

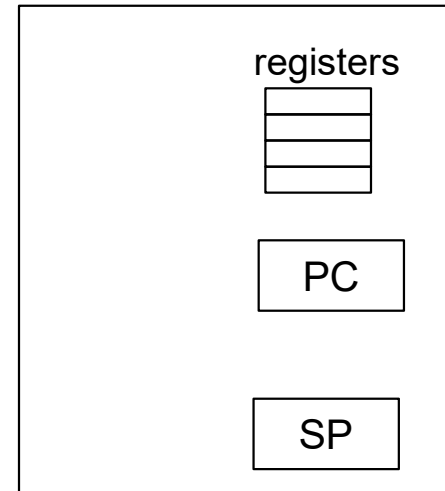
Thread context



Thread context



Thread context



CPU

- How to load registers?
- How to load stack pointer?
- How to resume execution?

Load state of the next thread (copy from memory to CPU)

- Linux: `setcontext()` copies context from memory to CPU
- How does the thread that was running (before the `setcontext`) resume?

Switching threads

- `getcontext()` copies context from CPU to memory
- `setcontext()` copies context from memory to CPU
- Does the following code work?

Thread A

```
do stuff
getcontext (saves state of thread A)
setcontext to thread B
do more stuff
```

Thread B

```
do stuff
getcontext (saves state of thread B)
setcontext to thread A
```

- After B calls `setcontext()`, where does A resume execution?
- Where do we want A to resume execution?
- Linux: `swapcontext()` correctly combines `getcontext()` and `setcontext()`

Who is carrying out these steps (e.g., switching from thread A to B)?

- The CPU?
- The OS?
- Which thread is carrying out these steps?

Example of thread switching

Thread 1

```
print "start thread 1"  
yield()  
print "end thread 1"
```

Thread 2

```
print "start thread 2"  
yield()  
print "end thread 2"
```

yield()

```
print "start yield: thread %d"  
switch to next thread (swapcontext)  
print "end yield: thread %d"
```

Thread 1 output

```
start thread 1  
start yield: thread 1
```

```
end yield: thread 1  
end thread 1
```

Thread 2 output

```
start thread 2  
start yield: thread 2
```

```
end yield: thread 2  
end thread 2
```

Example of thread switching

Thread 1

```
print "start thread 1"  
yield()  
print "end thread 1"
```

Thread 2

```
print "start thread 2"  
yield()  
print "end thread 2"
```

yield()

```
print "start yield: thread %d"  
switch to next thread (swapcontext)  
print "end yield: thread %d"
```

Thread 1 output

```
start thread 1  
start yield: thread 1  
  
end yield: thread 1  
end thread 1
```

Thread 2 output

```
start thread 2  
start yield: thread 2  
  
end yield: thread 2  
end thread 2
```

Example of thread switching

Thread 1

```
print "start thread 1"  
yield()  
print "end thread 1"
```

Thread 2

```
print "start thread 2"  
yield()  
print "end thread 2"
```

yield()

```
print "start yield: thread %d"  
switch to next thread (swapcontext)  
print "end yield: thread %d"
```

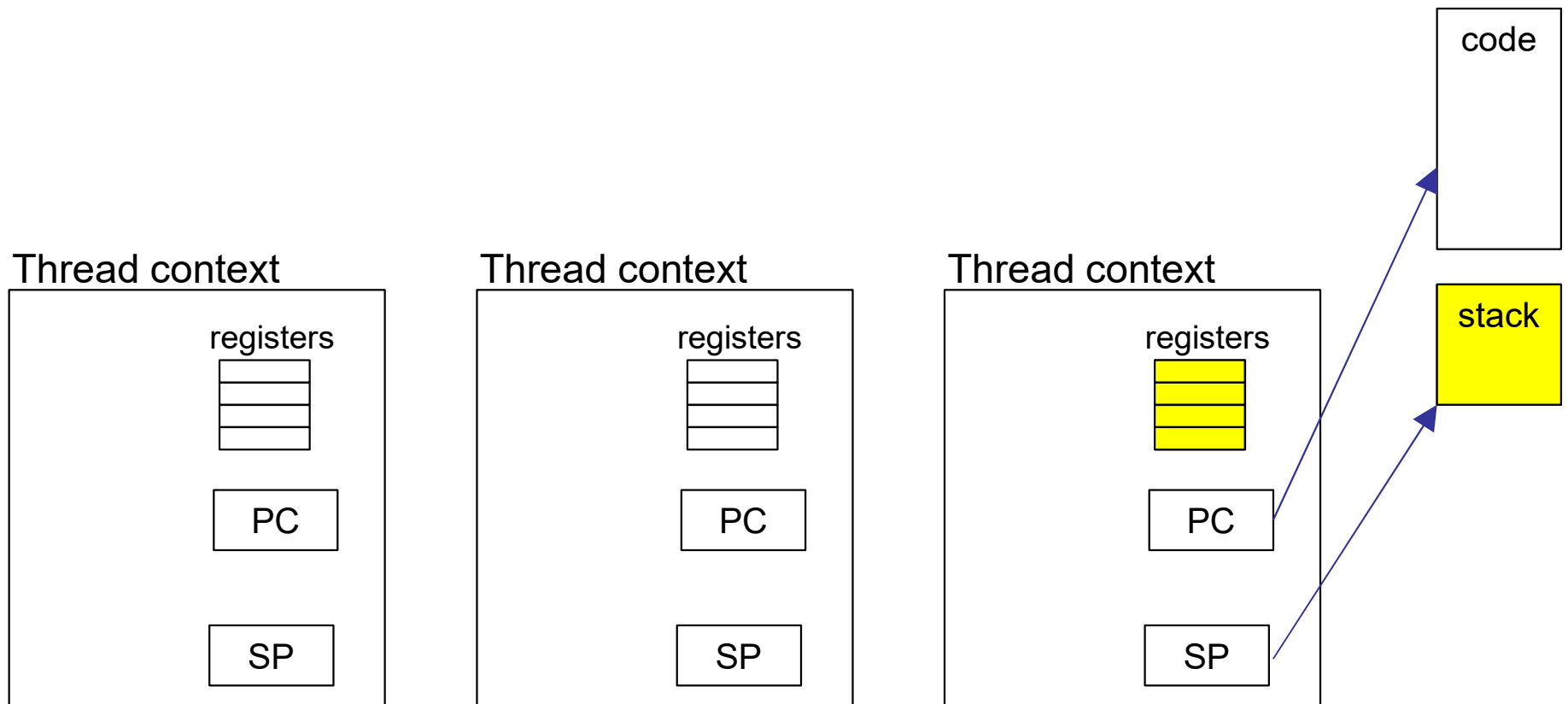
Thread 2 output

```
start thread 1  
start yield: thread 1  
start thread 2  
start yield: thread 2  
end yield: thread 1  
end thread 1  
end yield: thread 2  
end thread 2
```


Creating a new thread

- Challenging to create a running thread
- Instead, create **ready** thread
 - Key idea: make it look like it was running, put on ready queue
 - Then just wait for it to be scheduled!
- Construct TCB in the state it would be in if it paused at start of its initial function

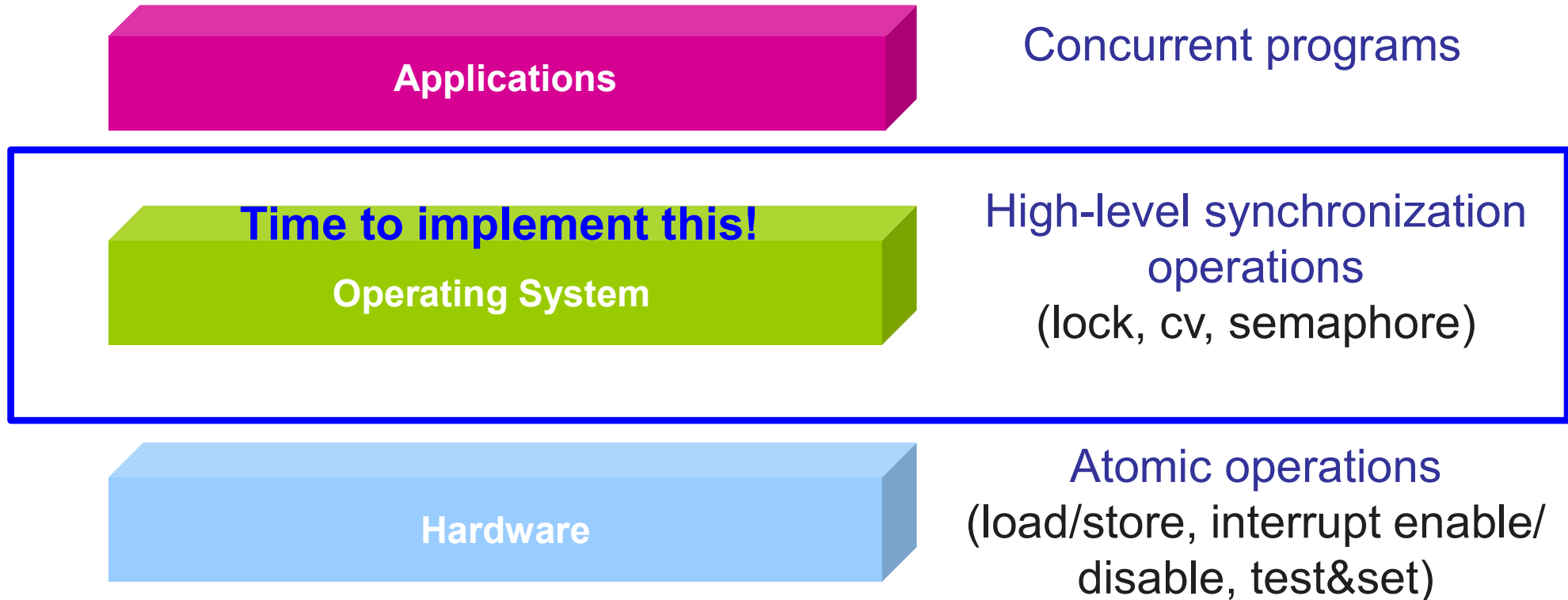
How to create a thread



How to create a thread

1. Allocate TCB
2. Allocate stack
3. Initialize context in TCB
 - New thread should look like it was about to call a function
 - Set PC to start of a function
 - Set general-purpose registers and stack to func parameters
 - Linux: `makecontext()` initializes context and stack
4. Add TCB to ready queue

High-level synchronization



Implementing high-level synchronization operations (e.g., lock)

- Uses shared data
 - ➔ The code that implements these operations must be **thread safe**
- Use synchronization (mutual exclusion, ordering) to implement synchronization ?!
 - Can't use the normal high-level synchronization operations
 - Instead, use atomic operations provided by hardware, e.g., atomic load, atomic store, etc.

Writing OS code

- Disadvantage
 - OS code can't use high-level synchronization operations, since it is implementing these
- Advantages
 - OS trusts its own code (OS doesn't trust user code)
 - OS controls the hardware (e.g., CPU)